目 录

一、系统功能设计

1.1 系统需求分析

1.2 数据流图

1.3 数据元素表

二、数据库概念模式设计

2.1 系统初步E-R图

2.2 系统基本E-R图

三、数据库逻辑模式设计与优化

3.1 数据库关系模式定义

3.2 关系模式范式等级的判定与规范化

3.3 数据库关系模式优化

四、数据库物理设计

4.1 存取方法

4.2 索引定义

一、系统功能设计

1.1 系统需求分析

1.1.1 应用环境与背景

在数字经济蓬勃发展的时代浪潮中，共享经济应运而生，成为资源优化配置与高效利用的创新典范。**共享经济**的核心在于打破传统所有权与使用权的固有绑定，通过数字化平台实现资源的动态流转与共享使用，从而提升资源的边际效用与社会整体福利。在高校这一独特而充满活力的微观社会生态系统中，随着教育教学模式的革新、科研创新活动的深度拓展以及校园文化生活的日益丰富多元化，我校学生群体对于各类高价值物品的需求呈现出鲜明的阶段性与场景化特征。一方面，数码产品、专业实验器材、高品质运动装备等物品因课程实验、学术竞赛、文体活动等特定场景需求而存在较高的使用频次，但另一方面，受限于学期时长、学业压力以及兴趣爱好变迁等因素，这些物品的实际使用周期往往较短，极易陷入长期闲置状态，造成资源浪费与经济成本的双重困境。

基于此，为了更好地整合和再利用闲置资源，提高资源配置效率，我们提出了**“智享北航：校园闲置资产共享租赁平台”**这一构想。



本项目聚焦于北航校园内部的 **C2C（Customer-to-Customer）共享租赁模式**构建与优化，致力于通过数字化技术手段实现对闲置物品资产的**精准管理**与**高效配置**。平台将采用高一致性事务处理机制，对物品信息（`items`）、租赁时间（`orders`）等关键数据进行实时同步与精准匹配，确保交易流程的流畅性与可靠性。同时，引入信用评价体系（`credit_logs`），以量化指标记录用户交易行为，激励诚信租赁，降低交易风险；并配套完善的仲裁机制（`arbitrations`），公正妥善处理租赁纠纷，为共享交易保驾护航。这一平台旨在激活北航师生的闲置资产潜能，提升资源利用效率，打造绿色、高效、便捷的校园共享经济生态，进而助力高校可持续发展与校园文化繁荣。

1.1.2 核心业务挑战

系统核心需解决的业务难题包括：

1. **高并发时间冲突与原子性控制：** 确保同一物品在任何时间段内只能被租借一次，这是共享系统的核心挑战。本项目将利用 **TaurusDB (for MySQL) 的事务特性**和 **`SELECT ... FOR UPDATE` 机制**，实现租赁预订。
2. **复杂状态流转与资金结算：** 订单流程涉及押金支付、租金支付、退款、罚款计算等多个环节。通过**订单状态机**（`orders.status`）和**订单结算金额**，确保每笔交易有迹可循。
3. **异常处理机制：** 针对物品损坏、迟还等纠纷，设立独立的**仲裁实体**（`arbitrations`），确保公平公正记录惩罚并更新用户信用分。

1.1.3 管理员相关功能

- **用户管理：** 查看所有用户信息，对违规用户进行封禁/解封（`users.is_active`）。
- **仲裁管理：** 处理处于 `pending` 状态的仲裁请求（`arbitrations`），填写 `decision` 和 `penalty_amount`，并关闭仲裁。
- **分类管理：** 新增、修改、删除物品分类（`categories`）。
- **内容管理：** 删除违规的物品（`items`）或评论（`reviews`）。

1.1.4 用户与物品功能

- **用户基础功能：** 用户注册、登录、修改个人信息（`password_hash`, `phone`, `email`, `avatar_url`）。
- **物品管理：** 发布新物品（`items`），设定标题、描述、校区（`campus`）、租金（`daily_rent`）、押金（`deposit`）和图片（`image_urls`）。编辑、更新、删除自己发布的物品状态（`available`, `rented`等状态）。
- **物品浏览与搜索：** 按分类（`cid`）、校区（`campus`)搜索和筛选物品。

1.1.5 租赁与交易功能

- **创建订单：** 租客选择物品和时间段（`start_time`, `end_time`），创建 `orders` 订单（状态为 `pending`）。
- **订单状态流转：** 使用平台币支付后，物主确认订单（`confirmed`）；租赁开始（`in_progress`）；租客归还，物主确认（`completed`）；或取消（`cancelled`）。
- **订单查询：** 用户可以查看自己作为租客（`renter_uid`）或物主（`owner_uid`）的所有订单。

1.1.6 信用、评价与仲裁功能

- **评价体系：** 订单完成后（`completed`），租客可进行评价（`reviews`），包括评分（`rating`）和评论（`comment`）。

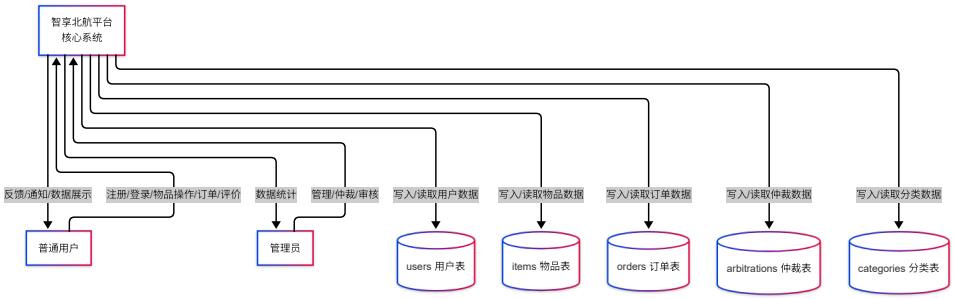
- **信用体系：** 用户的信用积分（`users.credit_score`）受 `credit_logs` 表驱动（例如，按时归还奖励、仲裁处罚扣分）。
- **纠纷仲裁：** 订单发生纠纷时（`orders.status` 变为 `dispute`），任何一方均可创建 `arbitrations` 记录，申请管理员介入。

1.1.7 消息通信功能

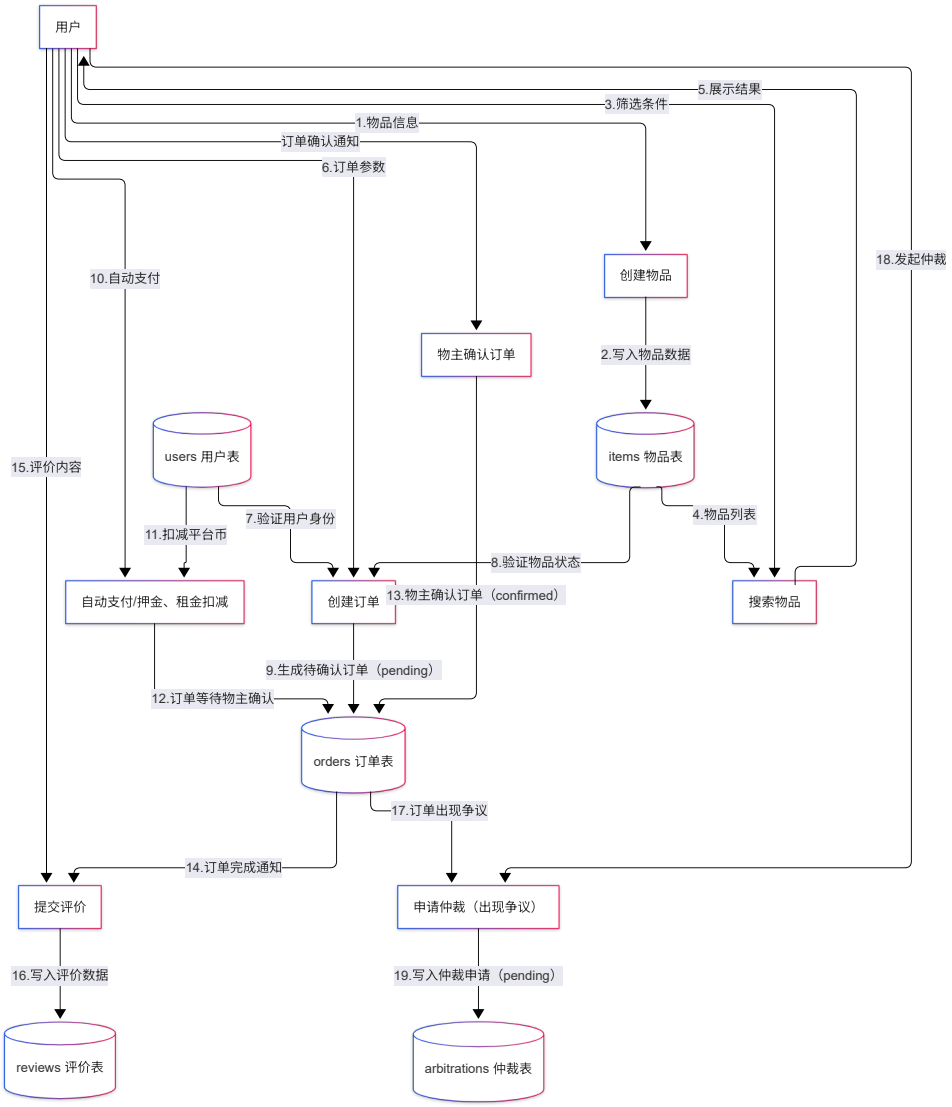
- **私信沟通：** 用户间可通过 `messages` 表（`sender_uid`，`receiver_uid`）进行点对点沟通（`message_type = 'text'`）。
- **系统通知：** 订单状态变更、仲裁结果等关键事件，由系统向用户发送通知（`message_type = 'notification'`）。

1.2 数据流图

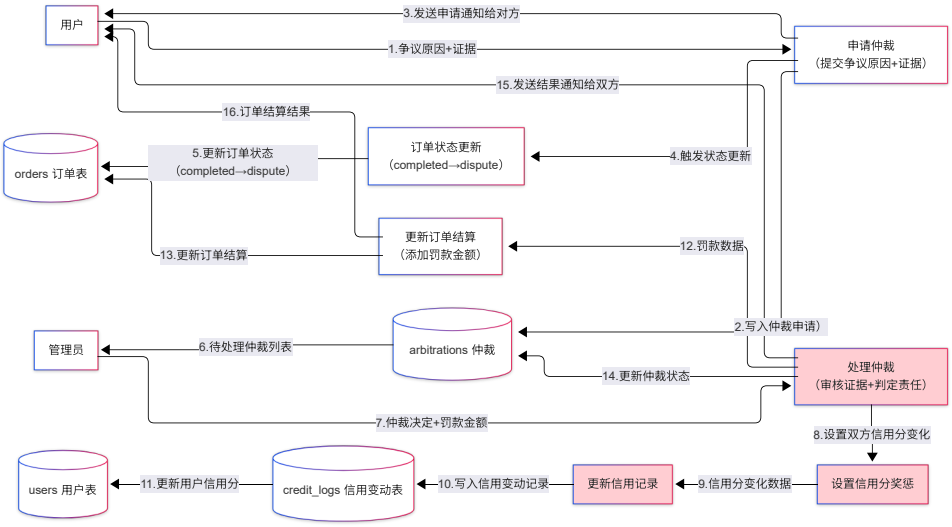
- **总体数据流图：** 描述了“普通用户”和“管理员”两类外部实体，与“智享北航平台”核心系统之间的数据交互。



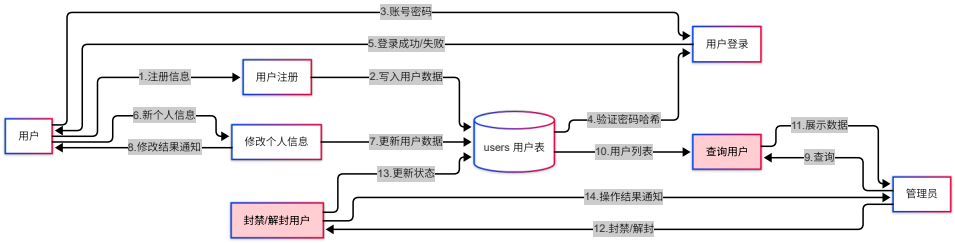
- **物品租赁数据流图：** 详细描述了用户从“创建物品”、“搜索物品”到“创建订单”、“支付结算”和“评价”的数据闭环。



- **纠纷仲裁数据流图：** 详细描述了订单进入 `dispute` 状态后，用户“申请仲裁”、管理员“处理仲裁”并最终影响 `credit_logs` 和 `orders` 结算的数据流。



- **用户管理数据流图：** 描述了用户注册、登录、修改信息，以及管理员封禁/解封用户的数据流。



1.3 数据元素表

1.3.1 用户表 (users)

字段名称	类型	备注
uid	INT	PK, 用户ID, AUTO_INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE, 用户名
email	VARCHAR(100)	NOT NULL, UNIQUE, 邮箱
password_hash	VARCHAR(255)	NOT NULL, 密码哈希
phone	VARCHAR(20)	手机号
coin	DECIMAL(10,2)	NOT NULL, 平台币余额
role	ENUM('student', 'admin')	NOT NULL, 角色
credit_score	INT	NOT NULL, 信用积分
avatar_url	VARCHAR(500)	头像链接
student_id	VARCHAR(20)	学号
is_active	BOOLEAN	是否激活（用于封禁）
created_at	TIMESTAMP	创建时间

1.3.2 分类表 (categories)

字段名称	类型	备注
cid	INT	PK, 分类ID, AUTO_INCREMENT
cname	VARCHAR(100)	NOT NULL, UNIQUE, 分类名称
description	TEXT	分类描述
parent_id	INT	FK, 关联 categories(cid), 支持层级分类
icon_url	VARCHAR(500)	分类图标
is_active	BOOLEAN	是否启用

1.3.3 物品表 (items)

字段名称	类型	备注
iid	INT	PK, 物品ID, AUTO_INCREMENT
owner_uid	INT	FK, 关联 users(uid), 所有者ID
cid	INT	FK, 关联 categories(cid), 分类ID
title	VARCHAR(200)	NOT NULL, 物品标题
description	TEXT	物品描述
campus	VARCHAR(50)	NOT NULL, 所在校区
daily_rent	DECIMAL(10,2)	NOT NULL, 日租金
deposit	DECIMAL(10,2)	NOT NULL, 押金
status	ENUM(...)	NOT NULL, 物品状态 (available, rented, ...)
image_urls	JSON	物品图片链接数组

1.3.4 租赁订单表 (orders)

字段名称	类型	备注
oid	INT	PK, 订单ID, AUTO_INCREMENT
iid	INT	FK, 关联 items(iid)
renter_uid	INT	FK, 关联 users(uid), 租客ID
start_time	DATETIME	NOT NULL, 租赁开始时间
end_time	DATETIME	NOT NULL, 租赁结束时间
actual_return_time	DATETIME	实际归还时间
status	ENUM(...)	NOT NULL, 订单状态 (pending, confirmed, ...)
final_total_amount	DECIMAL(10,2)	最终结算金额（含罚款）
penalty_amount	DECIMAL(10,2)	罚款金额

1.3.5 信用变动记录表 (credit_logs)

字段名称	类型	备注
clid	INT	PK, 记录ID, AUTO_INCREMENT
uid	INT	FK, 关联 users(uid)
type	ENUM(...)	变动类型 (penalty, reward, ...)
amount	INT	NOT NULL, 变动值（正负）
reason	VARCHAR(500)	变动原因
related_oid	INT	FK, 关联 orders(oid)
admin_uid	INT	FK, 关联 users(uid), 操作管理员ID
before_score	INT	NOT NULL, 变动前积分
after_score	INT	NOT NULL, 变动后积分

1.3.6 评论与评分表 (reviews)

字段名称	类型	备注
rid	INT	PK, 评论ID, AUTO_INCREMENT
oid	INT	FK, 关联 orders(oid), UNIQUE (一单一评)
reviewer_uid	INT	FK, 关联 users(uid), 评价者
reviewee_uid	INT	FK, 关联 users(uid), 被评价者
rating	INT	NOT NULL, 评分 (1-5)
comment	TEXT	评论内容
is_anonymous	BOOLEAN	是否匿名

1.3.7 消息表 (messages)

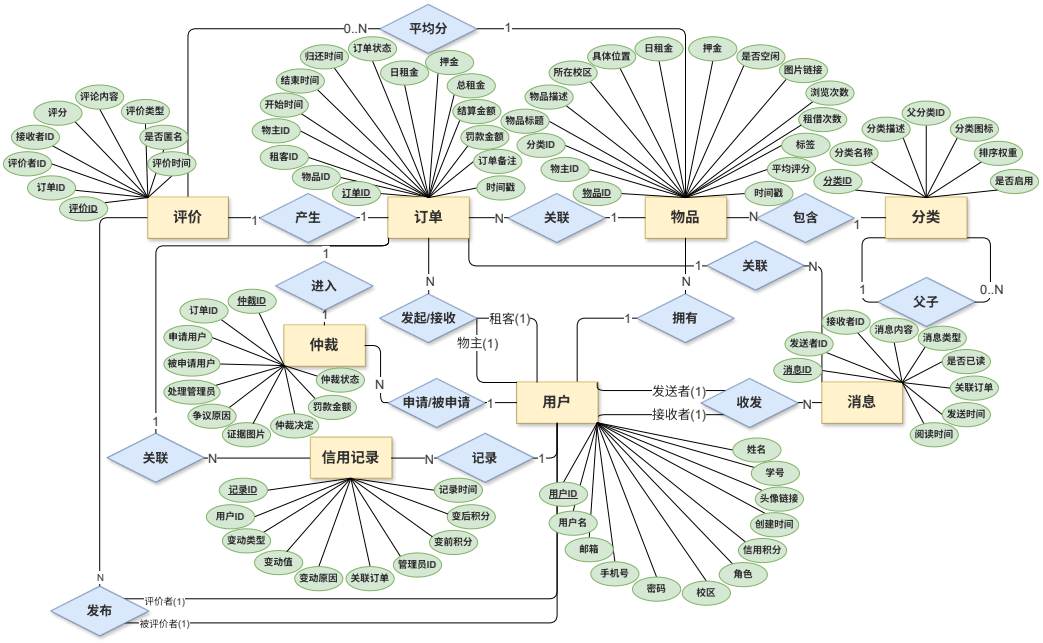
字段名称	类型	备注
mid	INT	PK, 消息ID, AUTO_INCREMENT
sender_uid	INT	FK, 关联 users(uid), 发送者
receiver_uid	INT	FK, 关联 users(uid), 接收者
content	TEXT	NOT NULL, 消息内容
message_type	ENUM('text', 'notification')	消息类型（私信/通知）
is_read	BOOLEAN	是否已读

1.3.8 仲裁记录表 (arbitrations)

字段名称	类型	备注
aid	INT	PK, 仲裁ID, AUTO_INCREMENT
oid	INT	FK, 关联 orders(oid), UNIQUE (一单一个仲裁)
applicant_uid	INT	FK, 关联 users(uid), 申请人
respondent_uid	INT	FK, 关联 users(uid), 被申请人
admin_uid	INT	FK, 关联 users(uid), 处理管理员
dispute_reason	TEXT	NOT NULL, 争议原因
evidence_urls	JSON	证据图片链接数组
decision	TEXT	仲裁决定
penalty_amount	DECIMAL(10,2)	罚款金额
status	ENUM(...)	仲裁状态 (pending, processing, resolved)

二、数据库概念模式设计

2.1 系统初步E-R图

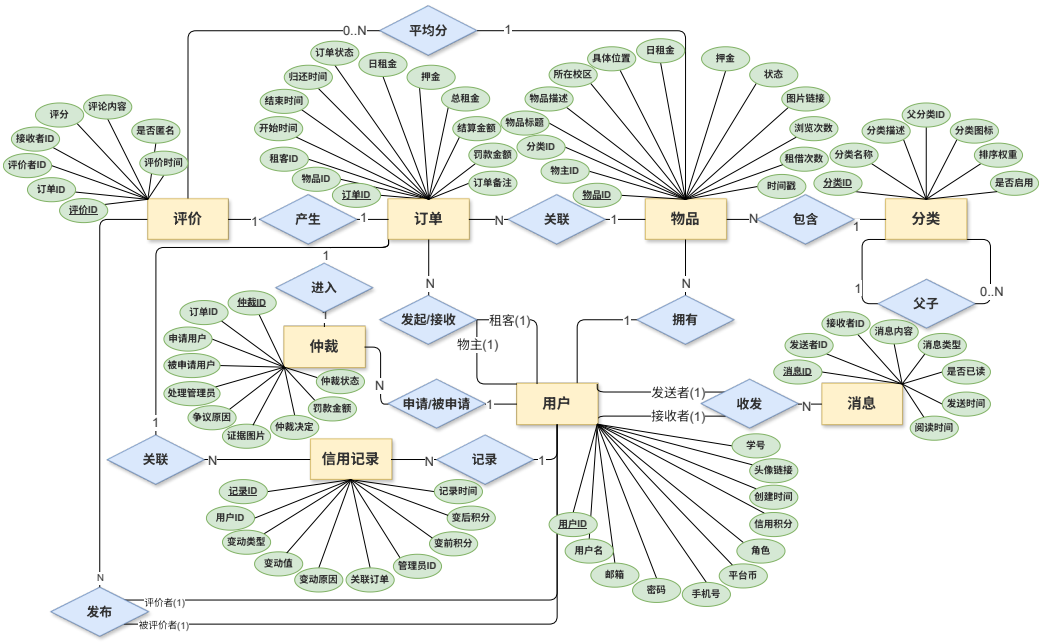


2.2 系统基本E-R图

2.2.1 E-R图优化过程

通过分析系统初步E-R图，我们围绕E-R 模型的“去冗余、弱耦合、强约束”做出了如下改进：

1. 将 `campus` 属性从用户剥离，删除 `real_name`、`review_type`、`tags` 等可推导或不满足 1NF 的属性；将物品可用性由 `is_available` 升级为 `status ENUM`，并补齐 `users.coin` 等核心业务字段；消息与订单解耦，仅保留 `sender`、`receiver` 与类型`type`(text/notification)等属性，允许下单前沟通。优化后模型更紧凑、约束更清晰，如下图所示：



三、数据库逻辑模式设计与优化

3.1 数据库关系模式定义

3.1.1 实体

1. **用户** `users` (**uid**, username, email, password_hash, phone, coin, role, credit_score, avatar_url, student_id, created_at, is_active)
2. **分类** `categories` (**cid**, cname, description, *parent_id*, icon_url, sort_order, is_active)
3. **物品** `items` (**iid**, *owner_uid*, *cid*, title, description, campus, location, daily_rent, deposit, status, image_urls, view_count, rent_count, created_at, updated_at)
4. **租赁订单** `orders` (**oid**, *iid*, *renter_uid*, start_time, end_time, actual_return_time, status, daily_rent, deposit, total_rent, final_total_amount, penalty_amount, remark, created_at, updated_at)
5. **信用变动记录** `credit_logs` (**clid**, *uid*, type, amount, reason, *related_oid*, *admin_uid*, before_score, after_score, created_at)
6. **评论与评分** `reviews` (**rid**, *oid*, *reviewer_uid*, *reviewee_uid*, rating, comment, is_anonymous, created_at)
7. **消息** `messages` (**mid**, *sender_uid*, *receiver_uid*, content, message_type, is_read, created_at, read_at)
8. **仲裁记录** `arbitrations` (**aid**, *oid*, *applicant_uid*, *respondent_uid*, *admin_uid*, dispute_reason, evidence_urls, decision, penalty_amount, status, created_at, resolved_at)

3.1.2 核心联系

- **拥有 (Owns/发布)**：用户(User) 与 物品(Item) 之间的 1:M 联系。
- **发起/接收 (Rents)**：用户(User) 与 租赁订单(Order) 之间的 1:M 联系（作为租客 renter_uid 和物主 owner_uid）。
- **关联 (Relates)**：物品(Item) 与 租赁订单(Order) 之间的 1:M 联系。
- **包含 (BelongsTo)**：物品(Item) 与 分类(Category) 之间的 M:1 联系。
- **父子 (Self-Relates)**：分类(Category) 实体内部的 1:M 自我联系（parent_id）。
- **产生 (Generates/评价)**：租赁订单(Order) 与 评论与评分(Review) 之间的 1:1 联系。
- **记录 (Logs)**：用户(User) 与 信用变动记录(CreditLog) 之间的 1:M 联系。
- **进入 (Drives)**：租赁订单(Order) 与 仲裁记录(Arbitration) 之间的 1:1 联系。
- **收发 (Sends/Receives)**：用户(User) 与 消息(Message) 之间的 1:M 联系（作为发送者 sender_uid 和接收者 receiver_uid）。

3.2 关系模式范式等级的判定与规范化

本项目所有关系模式均满足 **3NF**。

1. **满足 1NF：** 所有关系模式的属性都是**原子性**的，不可再分。对于 image_urls 和 evidence_urls 字段，我们使用了 JSON 类型，它在数据库层面被视为单个原子值（一个 JSON 文档），符合 1NF 的要求。

2. **满足 2NF：** 关系模式中没有部分函数依赖。

- 在本项目中，所有实体的主键（如 users.uid，items.iid，orders.oid 等）都是**单一属性**。
- 由于主键是单属性，所有非主属性都只能完全依赖于主键（候选键），不可能存在部分依赖于主键的情况。
- 因此，所有关系模式均满足 2NF。

3. **满足 3NF/BCNF 分析：** 关系模式中没有传递函数依赖（即非主属性不依赖于其他非主属性）。

- **users 表：** 主键 uid。username 和 email 均为 UNIQUE，是候选键。所有非主属性如 credit_score，role 均完全函数依赖于 uid（以及其他候选键）。所有决定因素（uid，username，email）都是超键。该模式满足 BCNF。
- **items 表：** 主键 iid。所有非主属性（如 title，campus，daily_rent）均完全函数依赖于 iid。owner_uid 和 cid 是外键，它们所代表的物主名称、分类名称等信息存储在各自的表中，避免了传递依赖。该模式满足 3NF/BCNF。

- **`orders` 表：** 主键 `oid`。所有非主属性（如 `start_time`, `status`, `final_total_amount`）均完全函数依赖于 `oid`。不存在非主属性依赖于其他非主属性的情况。该模式满足 3NF/BCNF。
- **`reviews` / `arbitrations` 表：** `oid` 在这两个表中均是 `UNIQUE` 外键，因此 `oid` 也是候选键。所有非主属性（如 `rating`, `decision`）均完全函数依赖于主键（`rid` / `aid`）和候选键（`oid`）。模式满足 BCNF。
- **`categories`，`credit_logs`，`messages` 表：** 所有非主属性均完全函数依赖于其各自的单属性主键，不存在传递依赖，均满足 3NF/BCNF。

3.3 数据库关系模式优化

在从初步设计到最终方案的迭代过程中，我们进行了以下优化：

1. **实体属性的精简与去冗余：**
- **`users` 表：** 移除了 `campus` 字段。分析发现，“校区”是物品（`items`）的固有属性（物品在哪里），而不是用户的属性（用户可能在两校区活动）。此修改消除了数据冗余，并使“按校区搜索物品”的逻辑更清晰。
- **`users` 表：** 移除了 `real_name` 字段，以简化用户信息，仅保留必要的 `student_id` 进行身份关联。
- **`reviews` 表：** 移除了 `review_type` 字段。该字段（“租客对物主”或“物主对租客”）完全可以通过 `reviewer_uid` 与 `orders` 表中的 `renter_uid` 和 `owner_uid` 比较得出，属于冗余数据，予以删除。
2. **状态与类型的规范化：**
- **`items` 表：** 将原有的 `is_available` (BOOLEAN) 字段 升级为 `status` (ENUM) 字段。`status` 包含 'available', 'rented', 'under_maintenance', 'removed' 等状态，能更精细地描述物品当前状态，而不是简单的“是否可用”，极大增强了业务逻辑的健壮性。
- **`items` 表：** 移除了 `tags` (VARCHAR) 字段。原计划使用逗号分隔，但这违反了 1NF。在 E-R 设计中，标签应作为 M:N 关系（`tags` 表）实现，为简化实体模型，暂时移除该功能，而不是采用反范式设计。
- **`item` 表：** 移除了 `avg_rating` 字段。该字段可通过聚合 `reviews` 表中的评分动态计算得出，避免数据冗余和更新异常，符合范式要求。
- **`orders` 表：** 移除了 `owner_uid` 字段。物主信息可通过关联 `items` 表的 `owner_uid` 获取，避免冗余存储，确保数据一致性。
3. **核心业务属性的调整：**
- **`users` 表：** 新增了 `coin` (DECIMAL) 字段。在系统初步E-R图中缺失了平台虚拟货币这一核心业务属性，最终版中予以添加，作为交易结算的基础。
- **`messages` 表：** 移除了 `related_oid` 字段。原设计试图将消息强绑定于订单，但优化后认为 `messages` 表应为通用通信模块，允许用户在订单创建前进行协商，因此解耦。
- **`messages` 表：** 移除了与订单的冗余联系，简化为仅包含发送者、接收者、内容和类型等基本属性。

- **`messages` 表：** 简化了 `message_type`，将 'system' 和 'notification' 合并为 'notification'，使类型定义更清晰。

四、数据库物理设计

物理设计是基于逻辑模式，为 GaussDB 数据库选择具体的存储结构和存取方法，以提高查询性能和保证数据完整性。

4.1 存取方法

在 `create_tables.sql` 中，我们所有表均使用了 `ENGINE=InnoDB`。

- **理由：** InnoDB 提供了**事务安全、行级锁定**和**外键约束**支持。
- **事务支持：** 对应本项目的高并发时间冲突，`START TRANSACTION`，`COMMIT`，`ROLLBACK` 是实现 `CheckAndCreateRentalOrder` 存储过程的基础。
- **存取结构：** InnoDB 默认使用 **B+ 树索引**（聚簇索引）来存储数据和索引，这为基于主键和索引的快速范围查询提供了高性能支持。

4.2 索引定义

我们为高频查询和连接字段（`WHERE` 子句、`JOIN` 操作）定义了以下索引：

1. **`users` 表：**
- `idx_username (username)` / `idx_email (email)`：优化登录和 `UNIQUE` 检查。
- `idx_credit_score (credit_score)`：优化信用排名。
2. **`items` 表：**
- `idx_owner_uid (owner_uid)`：优化查询“我发布的物品”。
- `idx_cid (cid)` / `idx_campus (campus)` / `idx_title (title)`：优化核心的物品搜索和筛选功能。
3. **`orders` 表：**
- `idx_renter_uid (renter_uid)`：优化“我的订单”列表查询。
- **核心索引：** **`idx_item_time (iid, start_time, end_time)`**
- **设计目的：** 这是本系统**最重要的物理设计**。它是一个复合索引，用于**高效执行时间冲突检测**。当查询特定 `iid` 的物品在 `[start_time, end_time]` 区间内是否被占用时，该索引可以极大地加速查询，避免全表扫描，是保障系统高并发性能的核心。
4. **`credit_logs` 表：**
- `idx_uid (uid)`：优化查询特定用户的信用记录。
5. **`messages` 表：**
- 复合索引 `idx_conversation (sender_uid, receiver_uid, created_at)`：优化私信会话列表的拉取和排序。
6. **`reviews` / `arbitrations` 表：**

- `idx_oid (oid)`：优化通过订单查询评价或仲裁的性能。